

MIDTERM PART 1

Part 01

Task 1.1

(base) anchu@Ans-MacBook-Air-4 ~ % cd ~/ism6562s26-class

git pull

Already up to date.

(base) anchu@Ans-MacBook-Air-4 ism6562s26-class % cd midterm/part01

(base) anchu@Ans-MacBook-Air-4 part01 % docker compose up -d

[+] Building 42.9s (16/16) FINISHED

```
=> [internal] load local bake definitions          0.0s
=> => reading from stdin 566B                      0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 410B                 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 7.0s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                   0.0s
=> => transferring context: 2B                      0.0s
=> [1/8] FROM docker.io/library/python:3.11-slim@sha256:9358444059ed78e2 6.8s
=> => resolve docker.io/library/python:3.11-slim@sha256:9358444059ed78e2 0.0s
=> => sha256:ccd4b4a613b5ce528e7385f571fafed3702a7390a0adbd68 249B / 249B 0.7s
=> => sha256:d6c44f24ac6d3ed8e0f59c89c5dd485f80db5404d 14.32MB / 14.32MB 3.2s
=> => sha256:b4822ed67dff6dee67342a10debf327642a6d8f6a65 1.27MB / 1.27MB 2.6s
=> => sha256:f4badebec24858ef2dc51256f6418328e305e9c3 30.14MB / 30.14MB 5.7s
=> => extracting sha256:f4badebec24858ef2dc51256f6418328e305e9c3c5a5e09 0.7s
=> => extracting sha256:b4822ed67dff6dee67342a10debf327642a6d8f6a65b7080 0.1s
=> => extracting sha256:d6c44f24ac6d3ed8e0f59c89c5dd485f80db5404d3710e52 0.3s
=> => extracting sha256:ccd4b4a613b5ce528e7385f571fafed3702a7390a0adbd685 0.0s
=> [internal] load build context                    0.0s
=> => transferring context: 10.22kB                 0.0s
=> [2/8] RUN apt-get update && apt-get install -y cron && rm -rf /var/l 17.2s
=> [3/8] RUN pip install --no-cache-dir psycpg2-binary 6.6s
=> [4/8] WORKDIR /app                              0.0s
=> [5/8] COPY etl_script.py .                      0.0s
=> [6/8] COPY crontab /etc/cron.d/etl-cron          0.0s
=> [7/8] COPY entrypoint.sh .                      0.0s
=> [8/8] RUN chmod 0644 /etc/cron.d/etl-cron && crontab /etc/cron.d/ 0.1s
=> exporting to image                              4.5s
=> => exporting layers                               3.6s
=> => exporting manifest sha256:91e4b858bd49b9b876cd3407b7bbac1eab981c82 0.0s
=> => exporting config sha256:699b5e88b077d2180ed1a96ecf9398f7d82840d8 0.0s
=> => exporting attestation manifest sha256:581eed25657e77a61ce8185a9396 0.0s
=> => exporting manifest list sha256:a8efb1129bd107154b5ace84b9ae69bcd5 0.0s
=> => naming to docker.io/library/part01-etl-service:latest 0.0s
=> => unpacking to docker.io/library/part01-etl-service:latest 0.8s
=> resolving provenance for metadata file          0.0s
```

[+] up 10/10

```
✓ Image part01-etl-service Built 43.1s
✓ Network part01_week03_network Created 0.0s
✓ Volume part01_warehouse_data Created 0.0s
✓ Volume part01_sales_data Created 0.0s
✓ Volume part01_hr_data Created 0.0s
✓ Container week03_warehouse_db Healthy 11.3s
✓ Container week03_sales_db Healthy 11.3s
✓ Container week03_hr_db Healthy 11.3s
✓ Container week03_pgadmin Created 0.1s
✓ Container week03_etl Created 0.1s
```

(base) anchu@Ans-MacBook-Air-4 part01 % docker compose up -d

[+] up 5/5

```
✓ Container week03_sales_db Healthy 0.6s
✓ Container week03_warehouse_db Healthy 0.6s
✓ Container week03_hr_db Healthy 0.6s
```

```

✓ Container week03_etl      Running      0.0s
✓ Container week03_pgadmin  Running      0.0s
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS              PORTS
week03_etl          part01-etl-service  "/entrypoint.sh"      etl-service 22 seconds ago Up 11 seconds
week03_hr_db        postgres:16          "docker-entrypoint.s..." hr-db      22 seconds ago Up 21 seconds (healthy)
0.0.0.0:5502->5432/tcp, [::]:5502->5432/tcp
week03_pgadmin      dpage/pgadmin4:8.13 "/entrypoint.sh"      pgadmin    22 seconds ago Up 11 seconds
0.0.0.0:8200->80/tcp, [::]:8200->80/tcp
week03_sales_db     postgres:16          "docker-entrypoint.s..." sales-db    22 seconds ago Up 21 seconds
(healthy) 0.0.0.0:5501->5432/tcp, [::]:5501->5432/tcp
week03_warehouse_db postgres:16          "docker-entrypoint.s..." warehouse-db 22 seconds ago Up 21 seconds
(healthy) 0.0.0.0:5503->5432/tcp, [::]:5503->5432/tcp
(base) anchu@Ans-MacBook-Air-4 part01 %

```

Task 1.2

```

(base) anchu@Ans-MacBook-Air-4 part01 % docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS              PORTS
week03_etl          part01-etl-service  "/entrypoint.sh"      etl-service 22 seconds ago Up 11 seconds
week03_hr_db        postgres:16          "docker-entrypoint.s..." hr-db      22 seconds ago Up 21 seconds (healthy)
0.0.0.0:5502->5432/tcp, [::]:5502->5432/tcp
week03_pgadmin      dpage/pgadmin4:8.13 "/entrypoint.sh"      pgadmin    22 seconds ago Up 11 seconds
0.0.0.0:8200->80/tcp, [::]:8200->80/tcp
week03_sales_db     postgres:16          "docker-entrypoint.s..." sales-db    22 seconds ago Up 21 seconds
(healthy) 0.0.0.0:5501->5432/tcp, [::]:5501->5432/tcp
week03_warehouse_db postgres:16          "docker-entrypoint.s..." warehouse-db 22 seconds ago Up 21 seconds
(healthy) 0.0.0.0:5503->5432/tcp, [::]:5503->5432/tcp
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec sales-db psql -U student -d sales -c '\dt'

```

List of relations

```

Schema | Name | Type | Owner
-----+-----+-----+-----
public | customers | table | student
public | orders | table | student
public | products | table | student
(3 rows)

```

```

(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec hr-db psql -U student -d hr -c '\dt'

```

List of relations

```

Schema | Name | Type | Owner
-----+-----+-----+-----
public | departments | table | student
public | employees | table | student
public | timesheets | table | student
(3 rows)

```

```

(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec warehouse-db psql -U student -d warehouse -c '\dt'

```

List of relations

```

Schema | Name | Type | Owner
-----+-----+-----+-----
public | dim_customer | table | student
public | dim_date | table | student
public | dim_employee | table | student
public | dim_product | table | student
public | fact_sales | table | student
(5 rows)

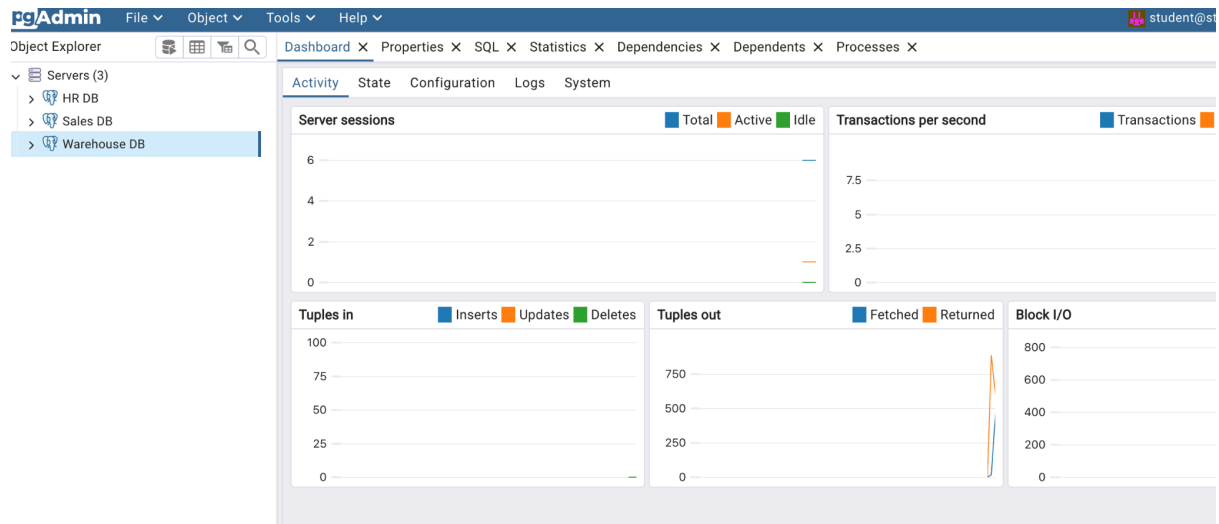
```

```

(base) anchu@Ans-MacBook-Air-4 part01 %

```

Task 1.3



Part 2

sales=# \d customers

\d products

\d orders

Table "public.products"

Column	Type	Collation	Nullable	Default
product_id	integer		not null	nextval('products_product_id_seq'::regclass)
product_name	character varying(200)		not null	
category	character varying(100)		not null	
unit_price	numeric(10,2)		not null	
created_at	timestamp without time zone			now()

Indexes:

"products_pkey" PRIMARY KEY, btree (product_id)

"idx_products_category" btree (category)

Referenced by:

TABLE "orders" CONSTRAINT "orders_product_id_fkey" FOREIGN KEY (product_id) REFERENCES products(product_id)

Table "public.orders"

Column	Type	Collation	Nullable	Default
order_id	integer		not null	nextval('orders_order_id_seq'::regclass)
customer_id	integer		not null	
product_id	integer		not null	
quantity	integer		not null	
total_price	numeric(10,2)		not null	
order_date	date		not null	

Indexes:

"orders_pkey" PRIMARY KEY, btree (order_id)

"idx_orders_customer_id" btree (customer_id)

"idx_orders_order_date" btree (order_date)

"idx_orders_product_id" btree (product_id)

Check constraints:

"orders_quantity_check" CHECK (quantity > 0)

Foreign-key constraints:

"orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)

"orders_product_id_fkey" FOREIGN KEY (product_id) REFERENCES products(product_id)

sales=# \d customers

Table "public.customers"

Column	Type	Collation	Nullable	Default
customer_id	integer		not null	nextval('customers_customer_id_seq'::regclass)

```

first_name | character varying(100) | | not null |
last_name  | character varying(100) | | not null |
email      | character varying(255) | | not null |
city       | character varying(100) | | |
state      | character varying(2)   | | |
created_at | timestamp without time zone | | | now()

```

Indexes:

```

"customers_pkey" PRIMARY KEY, btree (customer_id)
"customers_email_key" UNIQUE CONSTRAINT, btree (email)
"idx_customers_email" btree (email)
"idx_customers_state" btree (state)

```

Referenced by:

```

TABLE "orders" CONSTRAINT "orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES
customers(customer_id)
sales=#

```

Written Response

The sales schema demonstrates normalization by separating data into three tables: **customers**, **products**, and **orders**, where each table stores data about only one entity. The customers table stores customer information, the products table stores product information, and the orders table stores transaction data. This follows normalization because each table has a clear purpose.

Each table has a primary key that uniquely identifies each record. The primary keys are *customer_id* in customers, *product_id* in products, and *order_id* in orders. The orders table also contains foreign keys: *customer_id* references customers and *product_id* references products. These foreign keys create relationships and ensure that every order is linked to a valid customer and product.

The schema avoids data redundancy by not repeating customer or product details in the orders table. Instead of storing customer names or product names in every order, the orders table only stores their IDs. This means customer and product information is stored only once, which reduces duplication and makes updates easier.

Overall, the schema is normalized because each table stores one type of data, relationships are handled through keys, and duplicate data is minimized.

Task 2.2

```

(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec hr-db psql -U student -d hr
psql (16.13 (Debian 16.13-1.pgdg13+1))

```

Type "help" for help.

```
hr=# \d departments
```

```
\d employees
```

```
\d timesheets
```

```
hr=# \d departments
```

```
\d employees
```

```
\d timesheets
```

```

Table "public.timesheets"
Column | Type | Collation | Nullable | Default
-----+-----+-----+-----+-----
timesheet_id | integer | | not null | nextval('timesheets_timesheet_id_seq'::regclass)
employee_id | integer | | not null |
work_date | date | | not null |
hours_worked | numeric(4,2) | | not null |

```

Indexes:

```

"timesheets_pkey" PRIMARY KEY, btree (timesheet_id)
"idx_timesheets_employee_id" btree (employee_id)
"idx_timesheets_work_date" btree (work_date)
"timesheets_employee_id_work_date_key" UNIQUE CONSTRAINT, btree (employee_id, work_date)

```

Check constraints:

```
"timesheets_hours_worked_check" CHECK (hours_worked >= 0::numeric AND hours_worked <= 24::numeric)
```

Foreign-key constraints:

"timesheets_employee_id_fkey" FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
 hr=# \d departments

Table "public.departments"					
Column	Type	Collation	Nullable	Default	
department_id	integer		not null	nextval('departments_department_id_seq'::regclass)	
department_name	character varying(100)		not null		
location	character varying(100)		not null		

Indexes:

"departments_pkey" PRIMARY KEY, btree (department_id)

"departments_department_name_key" UNIQUE CONSTRAINT, btree (department_name)

Referenced by:

TABLE "employees" CONSTRAINT "employees_department_id_fkey" FOREIGN KEY (department_id)
 REFERENCES departments(department_id)

hr=# \d employees

Table "public.employees"					
Column	Type	Collation	Nullable	Default	
employee_id	integer		not null	nextval('employees_employee_id_seq'::regclass)	
first_name	character varying(100)		not null		
last_name	character varying(100)		not null		
email	character varying(255)		not null		
department_id	integer		not null		
hire_date	date		not null		
job_title	character varying(100)		not null		

Indexes:

"employees_pkey" PRIMARY KEY, btree (employee_id)

"employees_email_key" UNIQUE CONSTRAINT, btree (email)

"idx_employees_department_id" btree (department_id)

"idx_employees_hire_date" btree (hire_date)

Foreign-key constraints:

"employees_department_id_fkey" FOREIGN KEY (department_id) REFERENCES departments(department_id)

Referenced by:

TABLE "timesheets" CONSTRAINT "timesheets_employee_id_fkey" FOREIGN KEY (employee_id) REFERENCES
 employees(employee_id)

hr=#

Written Response:

The HR schema shows a hierarchical relationship between **departments**, **employees**, and **timesheets**. A department can have many employees through the *department_id* foreign key in the employees table. Similarly, an employee can have many timesheet records through the *employee_id* foreign key in the timesheets table. This creates a one-to-many relationship from departments to employees and from employees to timesheets.

The **UNIQUE(employee_id, work_date)** constraint ensures that each employee can only have one timesheet entry per day. This prevents duplicate records where an employee might accidentally have multiple entries for the same date. Without this constraint, duplicate entries could cause incorrect calculations of total hours worked or payroll errors.

This constraint improves data quality by enforcing consistency and accuracy in time tracking. It ensures that each workday is recorded only once per employee, preventing double counting and maintaining reliable workforce data for reporting and analysis.

Task 2.3

Sales database Queries

```
sales=# SELECT p.category,
COUNT(*) AS order_count,
SUM(o.total_price) AS total_revenue
FROM orders o
```

```

JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
category | order_count | total_revenue
-----+-----+-----
Electronics | 15 | 7199.81
Furniture | 7 | 2789.92
Books | 5 | 395.91
(3 rows)
sales=# SELECT c.first_name, c.last_name, c.state,
COUNT(*) AS order_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.state
ORDER BY order_count DESC;
first_name | last_name | state | order_count
-----+-----+-----+-----
Alice | Johnson | FL | 5
Carol | Williams | GA | 4
Bob | Smith | FL | 4
Eva | Davis | NC | 3
David | Brown | FL | 3
Frank | Miller | FL | 3
Grace | Wilson | TN | 3
Henry | Taylor | GA | 2
(8 rows)
sales=#

```

HR database Queries

```

hr=# SELECT d.department_name, d.location,
COUNT(*) AS employee_count
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name, d.location
ORDER BY employee_count DESC;
department_name | location | employee_count
-----+-----+-----
Human Resources | Tampa | 3
Marketing | Orlando | 3
Sales | Miami | 3
Engineering | Tampa | 3
(4 rows)
hr=# SELECT d.department_name,
SUM(t.hours_worked) AS total_hours,
ROUND(AVG(t.hours_worked), 1) AS avg_hours_per_entry
FROM timesheets t
JOIN employees e ON t.employee_id = e.employee_id
JOIN departments d ON e.department_id = d.department_id
WHERE t.work_date BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY d.department_id, d.department_name
ORDER BY total_hours DESC;
department_name | total_hours | avg_hours_per_entry
-----+-----+-----
Engineering | 158.00 | 7.9
Sales | 139.50 | 7.8
Marketing | 133.00 | 7.8
Human Resources | 131.00 | 7.7
(4 rows)

```

PART 3

Task 3.1

```
warehouse=# \d dim_date
\d dim_customer
\d dim_product
\d dim_employee
\d fact_sales
```

Table "public.dim_date"

Column	Type	Collation	Nullable	Default
date_key	integer		not null	
full_date	date		not null	
year	integer		not null	
quarter	integer		not null	
month	integer		not null	
month_name	character varying(20)		not null	
day_of_month	integer		not null	
day_of_week	integer		not null	
day_name	character varying(20)		not null	
is_weekend	boolean		not null	

Indexes:

"dim_date_pkey" PRIMARY KEY, btree (date_key)

"dim_date_full_date_key" UNIQUE CONSTRAINT, btree (full_date)

Referenced by:

TABLE "fact_sales" CONSTRAINT "fact_sales_date_key_fkey" FOREIGN KEY (date_key) REFERENCES dim_date(date_key)

Table "public.dim_product"

Column	Type	Collation	Nullable	Default
product_key	integer		not null	nextval('dim_product_product_key_seq'::regclass)
source_product_id	integer		not null	
product_name	character varying(200)		not null	
category	character varying(100)		not null	
unit_price	numeric(10,2)		not null	

Indexes:

"dim_product_pkey" PRIMARY KEY, btree (product_key)

"dim_product_source_product_id_key" UNIQUE CONSTRAINT, btree (source_product_id)

Referenced by:

TABLE "fact_sales" CONSTRAINT "fact_sales_product_key_fkey" FOREIGN KEY (product_key) REFERENCES dim_product(product_key)

Table "public.dim_employee"

Column	Type	Collation	Nullable	Default
employee_key	integer		not null	nextval('dim_employee_employee_key_seq'::regclass)
source_employee_id	integer		not null	
first_name	character varying(100)		not null	
last_name	character varying(100)		not null	
email	character varying(255)		not null	
job_title	character varying(100)		not null	
department_name	character varying(100)		not null	
department_location	character varying(100)		not null	

Indexes:

"dim_employee_pkey" PRIMARY KEY, btree (employee_key)

"dim_employee_source_employee_id_key" UNIQUE CONSTRAINT, btree (source_employee_id)

Table "public.fact_sales"

Column	Type	Collation	Nullable	Default
sale_key	integer		not null	nextval('fact_sales_sale_key_seq'::regclass)
date_key	integer		not null	
customer_key	integer		not null	
product_key	integer		not null	
source_order_id	integer		not null	

```

quantity    | integer    |      | not null |
unit_price  | numeric(10,2) |      | not null |
total_price | numeric(10,2) |      | not null |

```

Indexes:

```

"fact_sales_pkey" PRIMARY KEY, btree (sale_key)
"fact_sales_source_order_id_key" UNIQUE CONSTRAINT, btree (source_order_id)
"idx_fact_sales_customer_key" btree (customer_key)
"idx_fact_sales_date_key" btree (date_key)
"idx_fact_sales_product_key" btree (product_key)

```

Foreign-key constraints:

```

"fact_sales_customer_key_fkey" FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key)
"fact_sales_date_key_fkey" FOREIGN KEY (date_key) REFERENCES dim_date(date_key)
"fact_sales_product_key_fkey" FOREIGN KEY (product_key) REFERENCES dim_product(product_key)

```

warehouse=# \d dim_customer

Table "public.dim_customer"				
Column	Type	Collation	Nullable	Default
customer_key	integer		not null	nextval('dim_customer_customer_key_seq'::regclass)
source_customer_id	integer		not null	
first_name	character varying(100)		not null	
last_name	character varying(100)		not null	
email	character varying(255)		not null	
city	character varying(100)			
state	character varying(2)			

Indexes:

```

"dim_customer_pkey" PRIMARY KEY, btree (customer_key)
"dim_customer_source_customer_id_key" UNIQUE CONSTRAINT, btree (source_customer_id)

```

Referenced by:

```

TABLE "fact_sales" CONSTRAINT "fact_sales_customer_key_fkey" FOREIGN KEY (customer_key) REFERENCES
dim_customer(customer_key)
warehouse=#

```

Task 3.2 – Operational DB vs Data Warehouse

The operational databases (**sales-db** and **hr-db**) are designed for transaction processing and follow a normalized structure, while the **warehouse-db** is designed for analytics and follows a denormalized star schema. In the sales database, customer and product information are stored in separate normalized tables (**customers** and **products**) and must be joined with the **orders** table to perform analysis. In contrast, the warehouse stores this information in **dim_customer** and **dim_product**, which use surrogate keys and are optimized for fast analytical queries rather than transaction updates.

The **dim_employee** table in the warehouse includes *department_name* and *department_location* directly, while the HR database requires a join between employees and departments to get this information. The warehouse denormalizes this data to reduce joins and improve query performance. Since warehouses are read-heavy systems used for reporting, storing related attributes together makes queries faster and simpler for analysts.

The warehouse also includes a separate **dim_date** table with attributes such as *month_name*, *day_name*, and *is_weekend*. Instead of calculating these values repeatedly from order dates, the warehouse precomputes them to improve performance and simplify time-based analysis. This allows analysts to easily group sales by month, weekday, or weekend without complex date functions.

Overall, operational databases prioritize data integrity and efficiency for transactions, while the data warehouse prioritizes query speed and analytical usability.

Task 3.3 – Facts vs Dimensions

The **fact_sales** table is the fact table, while **dim_customer**, **dim_product**, **dim_employee**, and **dim_date** are dimension tables. This can be identified because the fact table contains measurable business metrics such as *quantity*, *unit_price*, and *total_price*, along with foreign keys referencing the dimension tables. In contrast, the dimension tables contain descriptive attributes such as customer names, product categories, and dates, which provide context for analysis.

The grain of the **fact_sales** table is one row per sales transaction (one order). Each row represents a single order with its associated customer, product, and date information. This is shown by the presence of *source_order_id*, which uniquely identifies each transaction.

One business question an analyst could answer is: **"Which product categories generate the most revenue by state and month?"** To answer this, the analyst would join **fact_sales** with **dim_product** (to get category), **dim_customer** (to get state), and **dim_date** (to get month). The fact table provides the revenue data, while the dimension tables provide the descriptive fields needed for grouping and filtering.

This structure allows flexible business analysis by combining measurable facts with descriptive dimensions.

Task 4.1

```
) anchu@Ans-MacBook-Air-4 part01 % docker compose exec etl-service cat /app/etl_script.py
```

```
#!/usr/bin/env python3
```

```
"""
```

ETL Pipeline: Operational Databases → Data Warehouse

ISM 6562 - Week 03 Assignment

This script extracts data from two operational databases (Sales and HR), transforms it into a star schema format, and loads it into a data warehouse.

Phases:

1. EXTRACT - Read rows from sales-db and hr-db
2. TRANSFORM - Generate dim_date; denormalize employee + department
3. LOAD - Upsert dimensions and insert facts (idempotent)

```
"""
```

```
import os
```

```
import sys
```

```
from datetime import date, timedelta
```

```
import psycopg2
```

```
from psycopg2.extras import execute_values
```

```
# -----
```

```
# Database connection helpers
```

```
# -----
```

```
def get_connection(prefix):
```

```
    """Create a database connection using environment variables."""
```

```
    return psycopg2.connect(
```

```
        host=os.environ[f'{prefix}_DB_HOST'],
```

```
        port=os.environ[f'{prefix}_DB_PORT'],
```

```
        dbname=os.environ[f'{prefix}_DB_NAME'],
```

```
        user=os.environ[f'{prefix}_DB_USER'],
```

```
        password=os.environ[f'{prefix}_DB_PASSWORD'],
```

```
    )
```

```
# -----
```

```
# EXTRACT phase
```

```
# -----
```

```
def extract_sales(conn):
```

```
    """Extract customers, products, and orders from the sales database."""
```

```
    cur = conn.cursor()
```

```
    cur.execute("SELECT customer_id, first_name, last_name, email, city, state FROM customers;")
```

```
    customers = cur.fetchall()
```

```
    print(f" Extracted {len(customers)} customers from sales-db")
```

```
    cur.execute("SELECT product_id, product_name, category, unit_price FROM products;")
```

```
    products = cur.fetchall()
```

```
    print(f" Extracted {len(products)} products from sales-db")
```

```
    cur.execute("""
```

```
        SELECT order_id, customer_id, product_id, quantity, total_price, order_date
```

```
        FROM orders;
```

```
    """)
```

```
    orders = cur.fetchall()
```

```
    print(f" Extracted {len(orders)} orders from sales-db")
```

```
    cur.close()
```

```
    return customers, products, orders
```

```

def extract_hr(conn):
    """Extract employees joined with departments from the HR database."""
    cur = conn.cursor()
    cur.execute("""
        SELECT e.employee_id, e.first_name, e.last_name, e.email,
               e.job_title, d.department_name, d.location
        FROM employees e
        JOIN departments d ON e.department_id = d.department_id;
    """)
    employees = cur.fetchall()
    print(f" Extracted {len(employees)} employees from hr-db")
    cur.close()
    return employees

# -----
# TRANSFORM phase
# -----

def generate_dim_date_rows(year):
    """Generate one row per day for the given year."""
    rows = []
    day_names = ["Monday", "Tuesday", "Wednesday", "Thursday",
                  "Friday", "Saturday", "Sunday"]
    month_names = ["January", "February", "March", "April", "May", "June",
                   "July", "August", "September", "October", "November", "December"]
    current = date(year, 1, 1)
    end = date(year, 12, 31)
    while current <= end:
        date_key = int(current.strftime("%Y%m%d"))
        rows.append((
            date_key,
            current,
            current.year,
            (current.month - 1) // 3 + 1, # quarter
            current.month,
            month_names[current.month - 1],
            current.day,
            current.weekday(),           # 0=Monday
            day_names[current.weekday()],
            current.weekday() >= 5,      # is_weekend
        ))
        current += timedelta(days=1)
    print(f" Generated {len(rows)} dim_date rows for {year}")
    return rows

# -----
# LOAD phase
# -----

def load_dim_date(cur, rows):
    """Load date dimension (insert, skip conflicts)."""
    execute_values(cur, """
        INSERT INTO dim_date (date_key, full_date, year, quarter, month,
                               month_name, day_of_month, day_of_week, day_name, is_weekend)
        VALUES %s
        ON CONFLICT (date_key) DO NOTHING;
    """, rows)
    print(f" Loaded dim_date ({len(rows)} rows, conflicts skipped)")

def load_dim_customer(cur, customers):
    """Load customer dimension (upsert on source_customer_id)."""
    rows = [(c[0], c[1], c[2], c[3], c[4], c[5]) for c in customers]
    execute_values(cur, """
        INSERT INTO dim_customer (source_customer_id, first_name, last_name, email, city, state)
        VALUES %s
        ON CONFLICT (source_customer_id) DO UPDATE SET
            first_name = EXCLUDED.first_name,
    """)

```

```

        last_name = EXCLUDED.last_name,
        email     = EXCLUDED.email,
        city      = EXCLUDED.city,
        state     = EXCLUDED.state;
    """ , rows)
    print(f" Loaded dim_customer ({len(rows)} rows, upserted)")
def load_dim_product(cur, products):
    """Load product dimension (upsert on source_product_id)."""
    rows = [(p[0], p[1], p[2], p[3]) for p in products]
    execute_values(cur, """
        INSERT INTO dim_product (source_product_id, product_name, category, unit_price)
        VALUES %s
        ON CONFLICT (source_product_id) DO UPDATE SET
            product_name = EXCLUDED.product_name,
            category     = EXCLUDED.category,
            unit_price   = EXCLUDED.unit_price;
    """ , rows)
    print(f" Loaded dim_product ({len(rows)} rows, upserted)")
def load_dim_employee(cur, employees):
    """Load employee dimension with denormalized department info."""
    rows = [(e[0], e[1], e[2], e[3], e[4], e[5], e[6]) for e in employees]
    execute_values(cur, """
        INSERT INTO dim_employee (source_employee_id, first_name, last_name,
                                   email, job_title, department_name, department_location)
        VALUES %s
        ON CONFLICT (source_employee_id) DO UPDATE SET
            first_name   = EXCLUDED.first_name,
            last_name    = EXCLUDED.last_name,
            email        = EXCLUDED.email,
            job_title    = EXCLUDED.job_title,
            department_name = EXCLUDED.department_name,
            department_location = EXCLUDED.department_location;
    """ , rows)
    print(f" Loaded dim_employee ({len(rows)} rows, upserted)")
def load_fact_sales(cur, orders):
    """Load fact_sales by looking up surrogate keys from dimensions."""
    inserted = 0
    skipped = 0
    for order in orders:
        order_id, customer_id, product_id, quantity, total_price, order_date = order
        date_key = int(order_date.strftime("%Y%m%d"))
        # Look up surrogate keys
        cur.execute("SELECT customer_key FROM dim_customer WHERE source_customer_id = %s;",
                    (customer_id,))
        cust_row = cur.fetchone()
        cur.execute("SELECT product_key FROM dim_product WHERE source_product_id = %s;",
                    (product_id,))
        prod_row = cur.fetchone()
        cur.execute("SELECT unit_price FROM dim_product WHERE source_product_id = %s;",
                    (product_id,))
        price_row = cur.fetchone()
        if cust_row and prod_row and price_row:
            cur.execute("""
                INSERT INTO fact_sales (date_key, customer_key, product_key,
                                         source_order_id, quantity, unit_price, total_price)
                VALUES (%s, %s, %s, %s, %s, %s, %s)
                ON CONFLICT (source_order_id) DO NOTHING;
            """ , (date_key, cust_row[0], prod_row[0],
                    order_id, quantity, price_row[0], total_price))
            if cur.rowcount > 0:
                inserted += 1
            else:

```

```

        skipped += 1
    else:
        print(f" WARNING: Could not find dimension keys for order {order_id}")
    print(f" Loaded fact_sales ({inserted} inserted, {skipped} already existed)")
# -----
# Main ETL orchestrator
# -----
def run_etl():
    """Execute the full ETL pipeline."""
    print("=" * 60)
    print("ETL Pipeline Starting")
    print("=" * 60)
    # --- EXTRACT ---
    print("\n[EXTRACT] Reading from operational databases...")
    sales_conn = get_connection("SALES")
    customers, products, orders = extract_sales(sales_conn)
    sales_conn.close()
    hr_conn = get_connection("HR")
    employees = extract_hr(hr_conn)
    hr_conn.close()
    # --- TRANSFORM ---
    print("\n[TRANSFORM] Preparing dimension data...")
    dim_date_rows = generate_dim_date_rows(2026)
    print(" Employee dimension: denormalized with department info (done in extract query)")
    # --- LOAD ---
    print("\n[LOAD] Writing to data warehouse...")
    wh_conn = get_connection("WAREHOUSE")
    cur = wh_conn.cursor()
    load_dim_date(cur, dim_date_rows)
    load_dim_customer(cur, customers)
    load_dim_product(cur, products)
    load_dim_employee(cur, employees)
    load_fact_sales(cur, orders)
    wh_conn.commit()
    cur.close()
    wh_conn.close()
    print("\n" + "=" * 60)
    print("ETL Pipeline Complete")
    print("=" * 60)
if __name__ == "__main__":
    try:
        run_etl()
    except Exception as e:
        print(f"ETL FAILED: {e}", file=sys.stderr)
        sys.exit(1)
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec etl-service cat /etc/cron.d/etl-cron
# Run the ETL pipeline every 5 minutes, sending output to Docker logs
*/5 * * * * cd /app && /usr/local/bin/python etl_script.py >> /proc/1/fd/1 2>&1
(base) anchu@Ans-MacBook-Air-4 part01 %

```

Transform, and Load. In the **Extract phase**, the script connects to the **sales database** to retrieve customers, products, and orders, running queries like `SELECT customer_id, first_name, last_name, email, city, state FROM customers;`, `SELECT product_id, product_name, category, unit_price FROM products;`, and `SELECT order_id, customer_id, product_id, quantity, total_price, order_date FROM orders;`. From the **HR database**, it extracts employee data joined with department information using a query that combines `employees` and `departments`.

During the **Transform phase**, the script prepares the data for the star schema. The `dim_date` table is populated by generating one row per day for the year, including attributes such as year, quarter, month, month_name, day_of_month,

day_of_week, day_name, and a boolean `is_weekend`. The `dim_employee` table is denormalized by embedding department name and location directly into each employee row, avoiding the need for joins in warehouse queries.

In the **Load phase**, dimension tables are loaded first, followed by the fact table `fact_sales`. The script ensures **idempotency** by using `ON CONFLICT` clauses in all inserts. For example, inserting into `dim_customer` will update existing rows if `source_customer_id` already exists, and `fact_sales` uses `ON CONFLICT (source_order_id) DO NOTHING` to skip duplicate orders.

The ETL runs automatically according to the cron schedule `*/5 * * * *`, which executes the pipeline **every five minutes**, keeping the warehouse consistently updated without creating duplicate data.

Task 4.2

```
part01 % docker compose exec sales-db psql -U student -d sales
psql (16.13 (Debian 16.13-1.pgdg13+1))
Type "help" for help.
sales=# -- Run on sales-db
SELECT 'customers' AS source_table, COUNT(*) FROM customers
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'orders', COUNT(*) FROM orders;
source_table | count
-----+-----
customers   |      8
products    |     10
orders      |     27
(3 rows)
sales=# \q
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec hr-db psql -U student -d hr
psql (16.13 (Debian 16.13-1.pgdg13+1))
Type "help" for help.
hr=# SELECT 'dim_customer' AS warehouse_table, COUNT(*) FROM dim_customer
UNION ALL SELECT 'dim_product', COUNT(*) FROM dim_product
UNION ALL SELECT 'dim_employee', COUNT(*) FROM dim_employee
UNION ALL SELECT 'dim_date', COUNT(*) FROM dim_date
UNION ALL SELECT 'fact_sales', COUNT(*) FROM fact_sales;
ERROR: relation "dim_customer" does not exist
LINE 1: ... 'dim_customer' AS warehouse_table, COUNT(*) FROM dim_custom...
                  ^
hr=#
```

Task 4.3

1.

```
sales=# -- Run on sales-db
INSERT INTO orders (customer_id, product_id, quantity, total_price, order_date)
VALUES (2, 1, 1, 1299.99, '2026-02-25');
INSERT 0 1
```

2.

```
sales=# -- Run on sales-db
SELECT * FROM orders ORDER BY order_id DESC LIMIT 3;
order_id | customer_id | product_id | quantity | total_price | order_date
-----+-----+-----+-----+-----+-----
28 | 2 | 1 | 1 | 1299.99 | 2026-02-25
27 | 7 | 10 | 1 | 349.99 | 2026-02-22
26 | 3 | 9 | 1 | 199.99 | 2026-02-20
(3 rows)
```

3.

```
sales=# docker compose exec etl-service python /app/etl_script.py
sales=# -- Run on warehouse-db
SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
ERROR: syntax error at or near "docker"
LINE 1: docker compose exec etl-service python /app/etl_script.py
      ^

sales=# -- Run on warehouse-db
SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
ERROR: relation "fact_sales" does not exist
LINE 3: FROM fact_sales f
      ^

sales=# -- Run on warehouse-db
SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
ERROR: relation "fact_sales" does not exist
LINE 3: FROM fact_sales f
      ^

sales=# \q
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec etl-service python /app/etl_script.py
=====
ETL Pipeline Starting
=====
[EXTRACT] Reading from operational databases...
  Extracted 8 customers from sales-db
  Extracted 10 products from sales-db
  Extracted 28 orders from sales-db
  Extracted 12 employees from hr-db
[TRANSFORM] Preparing dimension data...
  Generated 365 dim_date rows for 2026
  Employee dimension: denormalized with department info (done in extract query)
[LOAD] Writing to data warehouse...
  Loaded dim_date (365 rows, conflicts skipped)
  Loaded dim_customer (8 rows, upserted)
  Loaded dim_product (10 rows, upserted)
  Loaded dim_employee (12 rows, upserted)
  Loaded fact_sales (0 inserted, 28 already existed)
=====
ETL Pipeline Complete
=====
(base) anchu@Ans-MacBook-Air-4 part01 %
```

4.

```
sales=# \q
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec etl-service python /app/etl_script.py
```

```
=====
ETL Pipeline Starting
=====
```

```
[EXTRACT] Reading from operational databases...
  Extracted 8 customers from sales-db
  Extracted 10 products from sales-db
  Extracted 28 orders from sales-db
  Extracted 12 employees from hr-db
[TRANSFORM] Preparing dimension data...
  Generated 365 dim_date rows for 2026
  Employee dimension: denormalized with department info (done in extract query)
[LOAD] Writing to data warehouse...
  Loaded dim_date (365 rows, conflicts skipped)
  Loaded dim_customer (8 rows, upserted)
  Loaded dim_product (10 rows, upserted)
  Loaded dim_employee (12 rows, upserted)
  Loaded fact_sales (0 inserted, 28 already existed)
=====
```

```
ETL Pipeline Complete
=====
```

```
(base) anchu@Ans-MacBook-Air-4 part01 % \q
zsh: command not found: q
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec hr-db psql -U student -d hr
psql (16.13 (Debian 16.13-1.pgdg13+1))
Type "help" for help.
hr=# -- Run on warehouse-db
SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
ERROR: relation "fact_sales" does not exist
LINE 3: FROM fact_sales f
          ^
hr=#
```

PART 5

```
(base) anchu@Ans-MacBook-Air-4 part01 % docker compose exec warehouse-db psql -U student -d warehouse
psql (16.13 (Debian 16.13-1.pgdg13+1))
```

```
Type "help" for help.
```

```
warehouse=# SELECT
```

```
  d.month_name,
  d.year,
  COUNT(f.sale_key) AS total_orders,
  SUM(f.total_price) AS total_revenue
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY d.year, d.month, d.month_name
ORDER BY d.year, d.month;
month_name | year | total_orders | total_revenue
```

```
-----+-----+-----+-----
January   | 2026 |          15 |    5773.78
February  | 2026 |          13 |    5911.85
```

(2 rows)

```
warehouse=# SELECT
  c.first_name,
  c.last_name,
  COUNT(f.sale_key) AS order_count,
  SUM(f.total_price) AS total_spent
FROM fact_sales f
JOIN dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.customer_key, c.first_name, c.last_name
ORDER BY total_spent DESC;
first_name | last_name | order_count | total_spent
```

first_name	last_name	order_count	total_spent
Bob	Smith	5	2391.94
Alice	Johnson	5	2209.94
Eva	Davis	3	1979.96
Frank	Miller	3	1949.97
Carol	Williams	4	1737.93
David	Brown	3	739.95
Grace	Wilson	3	549.96
Henry	Taylor	2	125.98

(8 rows)

```
warehouse=# SELECT
  p.category,
  d.month_name,
  d.year,
  SUM(f.total_price) AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY p.category, d.year, d.month, d.month_name
ORDER BY p.category, d.year, d.month;
category | month_name | year | total_revenue
```

category	month_name	year	total_revenue
Books	January	2026	263.94
Books	February	2026	131.97
Electronics	January	2026	3699.89
Electronics	February	2026	4799.91
Furniture	January	2026	1809.95
Furniture	February	2026	979.97

(6 rows)

warehouse=#

SALES DATABASE

QUERY PLAN

```
Sort (cost=53.21..53.48 rows=110 width=258) (actual time=0.251..0.254 rows=3 loops=1)
  Sort Key: (sum(o.total_price)) DESC
  Sort Method: quicksort  Memory: 25kB
-> HashAggregate (cost=48.10..49.48 rows=110 width=258) (actual time=0.227..0.233 rows=3 loops=1)
  Group Key: p.category
  Batches: 1  Memory Usage: 24kB
  -> Hash Join (cost=12.47..38.58 rows=1270 width=234) (actual time=0.112..0.130 rows=28 loops=1)
    Hash Cond: (o.product_id = p.product_id)
```


-> Seq Scan on orders o (cost=0.00..22.70 rows=1270 width=20) (actual time=0.061..0.065 rows=28 loops=1)
 -> Hash (cost=11.10..11.10 rows=110 width=222) (actual time=0.029..0.030 rows=10 loops=1)
 Buckets: 1024 Batches: 1 Memory Usage: 9kB
 -> Seq Scan on products p (cost=0.00..11.10 rows=110 width=222) (actual time=0.011..0.015 rows=10 loops=1)
 Planning Time: 0.691 ms
 Execution Time: 0.511 ms
 (14 rows)
 (END)

WAREHOUSE DATABASE

QUERY PLAN

Sort (cost=30.86..30.87 rows=3 width=50) (actual time=0.305..0.307 rows=3 loops=1)
 Sort Key: (sum(f.total_price)) DESC
 Sort Method: quicksort Memory: 25kB
 -> HashAggregate (cost=30.80..30.84 rows=3 width=50) (actual time=0.138..0.140 rows=3 loops=1)
 Group Key: p.category
 Batches: 1 Memory Usage: 24kB
 -> Hash Join (cost=1.23..23.53 rows=970 width=26) (actual time=0.106..0.115 rows=28 loops=1)
 Hash Cond: (f.product_key = p.product_key)
 -> Seq Scan on fact_sales f (cost=0.00..19.70 rows=970 width=20) (actual time=0.018..0.020 rows=28 loops=1)
 -> Hash (cost=1.10..1.10 rows=10 width=14) (actual time=0.066..0.066 rows=10 loops=1)
 Buckets: 1024 Batches: 1 Memory Usage: 9kB
 -> Seq Scan on dim_product p (cost=0.00..1.10 rows=10 width=14) (actual time=0.051..0.053 rows=10 loops=1)
 Planning Time: 3.984 ms
 Execution Time: 0.572 ms
 (14 rows)
 (END)

Written Response: Comparison of Execution Plans:

The sales-db query performs a **sequential scan** on both **orders** and **products** and uses a **hash join** to aggregate revenue by category. It involves **one join** and has a planning time of ~0.7 ms and execution time of ~0.5 ms. The warehouse-db query also uses **sequential scans** on **fact_sales** and **dim_product** with a hash join, but benefits from **denormalized surrogate keys**, reducing join complexity. Planning time is slightly higher (~4 ms), but execution time is similar (~0.57 ms).

With **millions of rows**, the differences would become more pronounced. The warehouse's **star schema** minimizes the number of joins and allows faster aggregation because fact tables already contain keys, whereas the operational sales-db would require joining large normalized tables, increasing both I/O and CPU time for scans and joins.